

1.0 数据容器概述

数据容器

- 一种可以容纳多份数据的数据类型（容器），容纳的每一份数据称之为1个元素，每一个元素都可以是任意类型的数据，如：字符串、数字、布尔等。

属性

- 有序: 是否支持索引, 切片
- 重复
- 修改

方法 — 定增删改查

是否支持重复元素 是否可以修改 是否有序

列表 (list)	字符串 (str)	元组 (tuple)	集合 (set)	字典 (dict)
是	否	否	否	否
是	是	是	否	否
是	否	否	否	否

1.1 列表

列表-介绍

- 列表是数据容器中的一类，是一旦可以存储多个数据（元素）的。
- 定义：列表名称 = [元素1, 元素2, 元素3, 元素4, 元素5...]
- 特点：
 - 可以存储不同类型的元素
 - 元素有序、可以重复、元素可以修改

下标 索引

序列类型：容器中的元素按特定顺序排列的、可通过索引访问的容器类型称之为序列。

注意：从前向后（正向索引），下标从0开始；从后向前（反向索引），下标从-1开始。

print(s)输出整个列表;

删除:del s[1]; 容器中直接删除此元素;

列表索引不能超出范围,否则报错;

列表-切片

- 介绍：切片是指对操作的数据截取其中一部分的操作。列表、字符串、元组都支持切片操作。

语法：序列数据[开始索引:结束索引:步长]

- 不包含结束索引位置对应的元素(开始索引未指定默认为0；结束索引未指定默认为列表长度(直到列表末尾)；步长未指定默认为1)
- 索引采用正向、反向索引都可以
- 步长是选取间隔，默认步长为1

举例：

- s[0:5:1] 切片后的结果为 ["A", "C", "E", "B", "D"]
- s[0:5:2] 切片后的结果为 ["A", "E", "D"]

序列容器都可以进行切片操作;

列表-常见方法

- 列表的常见方法:

方法	作用	样例
append()	在列表的尾部追加元素	s.append(10086)
insert()	在指定索引之前, 插入该元素	s.insert(0, 92)
remove()	移除列表中第一个匹配到的值	s.remove(75)
pop()	删除列表中指定索引位置的元素 (如果未指定索引, 默认删除最后一个)	s.pop(2) / s.pop()
sort()	对列表进行排序 (列表元素的数据类型一致, 才可以进行排序)	s.sort()
reverse()	反转列表元素	s.reverse()

定义空列表:list1=[]

常用容器函数: 1. sum():求容器元素之和; 2. len():求容器元素数量; 3.min()/max():求容器最值;
4.in: 判断某一元素是否存在容器中,返回 bool 值;

```
num_list = [*num_list1, *num_list2]
```

:将两个 list 解包(获取每个元素)后,融合为新的列表 num_list;

```
num_list = num_list1 + num_list2
```

1. 如何快速合并两个列表?

- 使用+运算符直接合并
- 使用*进行解包操作(解包指将列表这一类数据容器解开成独立的元素)

```
num_list = num_list1 + num_list2  
num_list = [*num_list1, *num_list2]
```

2. 如何判断一个元素是否存在于列表之中?

- 可以使用in运算符, 语法为: 元素 in 列表
- 返回结果为布尔值, True表示存在, False表示不存在

列表推导式 创建列表:

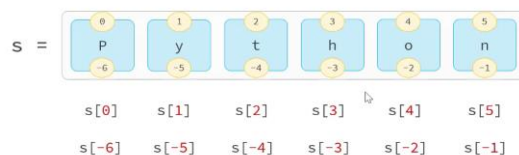
```
# 列表推导式 ---> 就是按照一定的规则快速生成一个列表的方法 --> 语法格式2: [要插入的值 for i in 序列/列表 if 条件]  
num_list = [12, 32, 45, 77, 80, 92, 33, 57, 97, 98, 110, 111, 122]  
new_list = [i**2 for i in num_list if i % 2 == 0]
```

1. 列表推导式

- 含义: 列表推导式就是按照一定规则快速生成一个列表的方法
- 格式1: 列表名称=[要插入列表的数据 for i in 列表]
- 格式2: 列表名称=[要插入列表的数据 for i in 列表 if 条件]

1.2 字符串

- 介绍: 字符串是字符的容器, 一个字符串中可以存放任意数量的字符。如: "Python" 'Python' """Python"""
- 特点: 不可变性(无法修改)、有序性、可迭代性。
- 字符串中的每一个字符元素都有其对应的下标(索引), 通过元素对应的索引, 就可以获取到对应的元素。

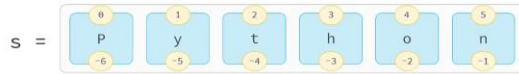


只能放字符,且不可修改;

字符串-切片

列表的切片是一致的

- 介绍：切片是指对操作的对象截取其中一部分的操作。字符串、列表、元组都支持切片操作。



- 语法：序列对象[开始索引:结束索引:步长]

- 不包含结束索引位置对应的元素(开始索引未指定默认为0; 结束索引未指定默认为-1; 步长未指定默认为1)
- 索引采用正向、反向索引都可以
- 步长是选取间隔，默认步长为1 (如果是-1，表示从后向前)

步长向右,起始为左右;步长向左,起始为右左;

- 字符串操作常用方法：

方法	作用	样例
<u>find()</u>	在字符串中查找子串，返回第一次出现的索引位置，找不到返回-1	s.find('Python')
count()	统计子串在字符串中出现的次数	s.count('H')
upper()	将字符串中的所有字母转换为大写	s.upper()
lower()	将字符串中的所有字母转换为小写	s.lower()
<u>split()</u>	将字符串按指定分隔符分割成列表	s.split(' ')
<u>strip()</u>	去除字符串两端的空白字符或指定字符	s.strip() / s.strip('*')
replace()	将字符串中的指定子串替换为新的子串	s.replace('H','C')
startswith()	检查字符串是否以指定子串开头，返回布尔值	s.startswith('P')

1.3 元组

元组

- 介绍：元组是**不可变**的序列，类似于列表，但创建后不能修改。

- 特点：

- 可以存储不同类型的元素
- 元素可以重复、有序、**不可以修改** (支持索引访问、切片)

- 定义：

```
# 定义元组
元组名称 = (元素1, 元素2, ...)
```

```
# 定义空元组
元组名称 = ()
元组名称 = tuple()
```

```
# 定义元组
t1 = (5, 7, 9, 1, 2, 3, 10, 6, 4, 8, 12, 7, 5)
```

```
# 定义空元组
t2 = ()
t3 = tuple()
```

- 方法：

- count(): 统计某元素在元组中出现的次数
- index(): 查找某个元素在元组中的索引位置(第一次出现的位置)

与列表最大区别:元素不可修改,只能访问;

定义单个元素的元组,元素后要跟逗号: s=(100,); 否则会是其他类型

元组-组包与解包

- 组包 (Packing) : 将多个值合并到一个容器 (元组、列表) 中。
- 解包 (Unpacking) : 将容器 (元组、列表) 解开成独立的元素, 分别赋值给多个变量。

```
# 定义元组, 组包
t1 = (5, 7, 9, 1)
# 定义元组, 组包
t2 = 5, 7, 9, 1

# 基础解包
a, b, c, d = t1

# (*)扩展解包
x, *y, z = t2 # x为5, y为[7, 9], z为1
s, *q = t2 # s为5, o为[7, 9, 1]
*o, e = t2 # o为[5, 7, 9], e为1
```

说明: 在元组解包时, * 表示收集剩余的所有元素, 允许我们处理不确定数量的元素 (生成列表, 以便于可以进行进一步的处理)。

多个元素解包后 生成列表;

多重元素就是 容器的嵌套;

元组的解包可以灵活应用:

```
for id,name,chinese,math,english in students:
    total = chinese + math + english
    avg = total / 3
```

1.4 集合

集合(set)

- 介绍: 集合(set)是一种无序的、不可重复、可修改的数据容器。
- 定义:

```
# 定义集合
s1 = {"C", "D", "X", "T", "O", "U"}

# 定义空集合
s2 = set()
```

与列表,元组最大区别:无序,不可重复

空集合定义 不可以使用{}表示空字典;由于集合无序, 因此是不支持下标索引访问

- 集合 (set) 中的常见方法:

操作	含义	样例
<u>add(..)</u>	添加元素到集合中	s1.add('t')
remove(..)	移除集合中的指定元素 (指定元素不存在将报错)	s1.remove('t')
pop()	随机删除集合中的元素并返回	e = s1.pop()
clear()	清空集合	s1.clear()
difference()	求取两个集合的差集 (包含在第一个集合但不包含在第二个集合的元素)	s1.difference(s2)
union()	求取两个集合的并集	s1.union(s2)
intersection()	求取两个集合的交集	s1.intersection(s2)

交集的写法 2: set = set1&set2 并集的写法: set= set1 | set2

差集的写法: set=set1 - set2

集合也存在推导式快速构建法:

```
# 方式三: 集合推导式 --> 快速构建集合, 语法: {要往集合中添加的数据 for s in set1 if 条件}
fb_set3 = {s for s in football_set if s not in basketball_set}
```

1.5 字典

字典

- 字典：使用键值对 (key: value) 来存储数据，每一个键都对应一个值，通过键 (key) 可以快速找到对应的值 (value)。
- 特点：键值对 (key: value) 存储、键 (key) 不能重复、可修改。
- 定义：

```
# 定义字典
字典名称 = {key: value, key:value, key:value ...}

# 定义空字典
字典名称 = {}

字典名称 = dict()

# 根据key获取value
值 = 字典名称[key]
```

```
# 定义字典
dict1 = {"王林": 675, "李慕婉": 608, "许立国": 478, ...}

# 定义空字典
dict2 = {}

dict3 = dict()

# 根据key获取value
score = dict1["李思"]
```

字典 (dict) 中的value可以是任何类型的数据，而key不能为可变类型 (如：不能为 列表list、集合set、字典dict)。

3. 字典的注意事项

value可以是任意类型，而key必须是不可变类型 (不能为list、set、dict)

字典内的key不允许重复，如果重复定义，后面的覆盖前面的

- 字典是没有索引下标的，不能根据索引获取值，只可以根据key获取value

- 字典的增删改查操作方式如下：dict1 = {"小智": 675, "李思": 608, "李琪": 478, "小黑": 545, "温韬": 429}

类型	操作	含义	样例
添加	字典名称[key] = value	往指定字典中添加key-value键值对	dict1["涛哥"] = 688
删除	字典名称.pop(key)	删除字典中指定的key，并返回该key对应的value	score = dict1.pop("涛哥")
	del 字典名称[key]	删除字典中指定的键值对	del dict1["涛哥"]
修改	字典名称[key] = value	修改字典中指定的key对应的值	dict1["小智"] = 658
查询	字典名称[key]	根据key获取value	dict1["涛哥"]
	字典名称.get(key)	根据key获取value	dict1.get("涛哥")
	字典名称.keys()	获取所有的key	dict1.keys()
	字典名称.values()	获取所有的value	dict1.values()
	字典名称.items()	获取所有的key-value键值对	dict1.items()

```
# 遍历
for k in dict1.keys():
    print(f"{k} : {dict1[k]}")

for item in dict1.items():
    print(f"{item[0]} : {item[1]}")

for k,v in dict1.items():
    print(f"{k} : {v}")
```

遍历方式;

.keys 和.values 返回列表;.items 返回列表中为元组的形式;

容器总结:

数据容器总结与对比					
特性	字符串 (str)	列表 (list)	元组 (tuple)	集合 (set)	字典 (dict)
有序性	有序	有序	有序	无序	有序(3.7+)
重复元素	允许	允许	允许	不允许	key不允许
可变性	不可变	可变	不可变	可变	可变
索引访问	支持	支持	支持	不支持	不支持
切片操作	支持	支持	支持	不支持	不支持
使用场景	文本处理	有序可重复数据集	固定数据记录	去重数据集	键值对

py 中没有块级作用域,只有函数,类级作用域;
列表,元组,集合,字典的 value 都允许存放不同类型数据;

2.0 函数

2.1 函数基础

函数

- 函数是组织好的、可重复使用的、用来实现特定功能的代码片段。
- 函数定义与调用：

```
# 定义函数
def 函数名(参数列表):
    函数体
    .....
    return 返回值

# 调用函数
函数名(参数)
```

```
# 定义函数
def out_line():
    print('-----')

# 调用函数
out_line()
```

接下来呢直接打开pay charm来演示一下呢

注意：函数定义时的参数列表与返回值语句是可有可无的(由需求确定)，函数必须先定义后调用执行（函数定义时，内部代码并未执行，在调用时才执行）。

2. 函数使用的注意事项
- 函数必须先定义，在调用
 - 函数定义时，并不会执行，只有在调用函数时，函数体的逻辑才会运行
 - 函数中通过缩进来描述归属关系

```
def circle_area_len(r):| 1 个用法
    return round(3.14 * r * r, 1), round(2 * 3.14 * r, 1)

al = circle_area_len(10)
print(al)
print(type(al))
```

py 中 return 可以一次返回多个值(组包为元组),使用时可以进行解包使用;
小数保留精度函数: Round(num,5): num 保留 5 位;

函数的说明文档

- 函数的说明文档 (Docstring) 是写在函数开头, 用三个引号包裹的字符串, 用于解释函数的功能、参数、返回值等信息, 方便调用者清楚函数的具体作用及细节。

```

# 定义一个函数, 根据半径, 计算圆的周长、面积
def circle_area_len(r):
    """
    该函数用于根据圆的半径, 计算圆的面积和圆的周长
    :param r: 圆的半径
    :return: 圆的面积, 圆的周长
    """
    return 3.14 * r * r, 2 * 3.14 * r

al = circle_area_len(10)
print(al)

```

描述函数的返回值

描述函数的参数

查看函数说明文档:

- 使用help函数, 比如: `help(circle_area_len)`
- 鼠标悬停在函数上, 自动展示 (推荐)

```

al = circle_area_len(10)
print(al)

```

PEP 8: E305 expected 2 blank lines after class or function definition, found 1

Reformat the file Alt+Shift+Enter More actions... Alt+Enter

0: /py_code_workspace/py_project1/04. 函数.py

def circle_area_len(r: Any) -> tuple[float | Any, float | Any]

该函数用于根据圆的半径, 计算圆的面积和圆的周长

Params: r - 圆的半径

Returns: 圆的面积, 圆的周长

加上对应的说明文档来

记住: 好的文档, 能让你的代码更容易理解、使用和维护!

可以通过 `help(函数名)` 将函数的说明直接打印出来;
鼠标悬停在函数调用处 可以看到函数的说明文档;

py 中函数的调用也是栈; 注意看 pycharm 调试中的栈; 符合后进先出的 都可以用栈;

2.2 函数进阶

2.2.1 函数变量的作用域

- 变量的作用域指的是变量的作用范围 (标识这个变量在哪里可以使用, 在哪儿不可以使用)。

```

# 定义函数
num = 100
def circle_area(r):
    pi = 3.14
    area = pi * r * r
    return area

count = 0
# 调用函数
c_area = circle_area(10)
print(c_area)

```

全局变量

局部变量

全局变量

在外面是无法访问的好

全局变量: 在函数之外定义的变量, 称之为全局变量, 在整个文件中 (包括函数内) 都可以使用 (通常定义在文件的顶部)。

局部变量: 在函数内部定义的变量, 称之为局部变量, 只能在该函数内部使用, 外部无法访问 (函数执行完毕后, 会自动销毁其内部局部变量)。

py 无主程序入口, 故不在函数中定义的变量都是全局变量;

global关键字

- `global` 关键字用于明确的告诉Python解释器, 在函数中要使用全局变量, 使得可以在函数内部修改全局变量的值。

```

num1 = 1 # 全局变量
def fun1():
    num1 = 100 # 局部变量
    print(num1)

fun1() # 100
print(num1) # 1

```

```

num1 = 1 # 全局变量
def fun1():
    global num1
    num1 = 100 # 修改全局变量num1
    print(num1)

fun1() # 100
print(num1) # 1

```

告诉python解释器, 函数中使用全局变量num1

函数中的 `global` 变量要先声明后使用;

3. 注意事项

- 尽量避免在函数中使用全局变量，因为会使代码难以维护和调试
- 考虑使用函数参数和返回值来传递数据，而不是依赖全局变量
- global主要用在程序的状态、配置和计数器等场景中

2.2.2 函数参数详解

2.2.2.1 传参方式

- 传参方式指的是，在调用函数时，传递实参的方式。

1. 位置参数：调用函数时根据函数定义时的位置来传递参数。

```
# 定义函数
def reg_stu(name, age, gender, city):
    print(f"注册成功,姓名:{name}, 年龄:{age}, 性别:{gender}, 城市:{city}")
    return {"name": name, "age": age, "gender": gender, "city": city}

# 调用函数
stu = reg_stu("张三", 18, "男", "北京")
print(stu)
```

```
# 定义函数
def 函数名(参数列表):
    函数体
    .....
    return 返回值

# 调用函数
函数名(参数...)
```

要求：调用函数时参数顺序与定义函数时参数顺序完全一致

2. 关键字参数：调用函数时以函数定义时形参名称作为关键字，以 "键=值" 的形式来传递参数(不要求顺序)。

```
# 定义函数
def reg_stu(name, age, gender, city):
    print(f"注册成功,姓名:{name}, 年龄:{age}, 性别:{gender}, 城市:{city}")
    return {"name": name, "age": age, "gender": gender, "city": city}

# 调用函数
stu = reg_stu(name="张三", age=18, gender="男", city="北京")
print(stu)

stu2 = reg_stu(gender="男", name="王武", city="上海", age=22)
print(stu2)

stu2 = reg_stu("赵四", 28, gender="男", city="上海")
print(stu2)
```

要求：如果位置参数与关键字参数混用，关键字参数必须在位置参数之后(关键字参数之间，没有顺序要求)

少于 3 个参数用位置;多于用关键字;

2.2.2.2 默认参数

- 默认参数也称为缺省参数，用于在定义函数时，为参数提供默认值，调用函数时，可以不传递有默认值的参数。

```
# 定义函数
def reg_stu(name, age, gender, city="北京"):
    print(f"注册成功,姓名:{name}, 年龄:{age}, 性别:{gender}, 城市:{city}")
    return {"name": name, "age": age, "gender": gender, "city": city}

# 调用函数
stu = reg_stu("张三", 18, "男")
print(stu)

stu = reg_stu("赵四", 22, "男", "深圳")
print(stu)
```

注意：默认参数必须放在没有默认值的参数列表的后面，一个函数在定义时是可以设置多个默认参数的。

注意：函数调用时，如果为默认参数传递了值，则会修改默认的参数值；如果没有传递该参数，则直接使用默认值。

2.2.2.3 不定长参数

不定长参数

- 介绍：不定长参数也叫可变参数，用于函数定义及调用时参数个数不确定（0个或多个）的场景。
- 类型：
 - 位置传递
 - 关键字传递

不定长参数-位置传递(*args)

```
# 定义函数
def calc_data(*args):
    min_data = min(args)
    max_data = max(args)
    avg_data = sum(args) / len(args)
    return min_data, max_data, round(avg_data, 1)

# 调用函数
data = calc_data(10, 20, 30, 40, 50, 60, 70, 80, 90, 100)
print(data)

data = calc_data(100, 200, 300, 400, 500)
print(data)
```

注意：传递的所有匹配的位置参数都会被args变量收集，这些参数会合并封装为一个元组，args是元组类型（注意并不会封装关键字参数）。

注意：args只是约定俗成的变量名，并不是关键字，这里可以使用任何合法的变量名（如 *data）。

不定长参数-关键字传递(**kwargs)

```
# 定义函数
def calc_data(*args, **kwargs):
    min_data = min(args)
    max_data = max(args)
    avg_data = sum(args) / len(args)

    if kwargs.get('round'):
        avg_data = round(avg_data, kwargs.get('round'))

    return min_data, max_data, avg_data

data = calc_data(100, 200, 300, 400, round=2, count=0)
print(data)

data = calc_data(33, 11, 28, 91, 32, 75, 49)
print(data)
```

注意：参数是以“键=值”形式传递的关键字参数，这些“键=值”参数都会被kwargs接受，并合并封装为一个字典类型。

注意：kwargs只是约定俗成的变量名，并不是关键字，这里可以使用任何合法的变量名（如 **options）。

位置参数在前,关键字参数在后;

3. *args与**kwargs的应用场景?

- *args适用于处理数量不确定的数据
- **kwargs适用于处理数量不确定的选项（函数的配置参数，用来定制函数的行为）

```
# 核心数据：你要什么
点奶茶("珍珠奶茶")

# 选项：你要什么样的
点奶茶("珍珠奶茶", 甜度="少糖", 冰度="去冰", 加料=["布丁", "珍珠"], 大小="大杯")
```

2.2.2.4 特殊参数

函数的参数类型

- 普通参数：数字、布尔、字符串、列表、元组、集合、字典等。
- 特殊参数：函数。

```
def circle_area(r):  
    area = 3.14 * r ** 2  
    return area  
  
area = circle_area(10)  
print(area)
```

```
def calc_score(score_list):  
    max_s = max(score_list)  
    min_s = min(score_list)  
    avg_s = round(sum(score_list) / len(score_list), 1)  
    return max_s, min_s, avg_s  
  
s_list = [589, 609, 605, 643, 677, 455, 477, 489, 503]  
max_score, min_score, avg_score = calc_score(s_list)
```

```
def add(x, y):  
    return x+y  
  
def subtract(x, y):  
    return x-y  
  
def calc(x, y, oper):  
    return oper(x, y)  
  
result = calc(10, 20, add)  
print(result)
```

传递的是函数中封装的逻辑

传递的是实际要计算的数据

2.2.2.5 匿名函数(lambda 表达式)

匿名函数

- 匿名函数指的是没有名称的函数，需要通过lambda表达式来声明函数，可以简化简单函数的编写（单行表达式）。

```
# 定义命名函数  
def 函数名(参数列表):  
    函数体...  
  
def out_line():  
    print('-----')  
  
def add(x, y):  
    return x + y  
  
out_line()  
print(add(10, 20))
```

命名函数

```
# 定义匿名函数  
lambda 参数列表 : 函数体  
  
out_line = lambda : print('-----')  
add = lambda x, y: x + y  
  
out_line()  
print(add(100, 200))
```

匿名函数

注意：函数逻辑比较简单（单行表达式）且只在一个地方使用时，可以考虑使用匿名函数，简化书写（通常作为高阶函数的参数使用）。

注意：匿名函数中可以返回结果，也可以不返回结果。返回结果时，不需要写return，表达式的运行结果就是要返回的结果。

`ls.sort(key=lambda item:len(item),reverse=True)`
Key 要求返回排序依据;item 是列表类中可以直接获取的每个项;
用 lambda 表达式,返回每个项的长度,即实现根据项长度来对 ls 进行排序;

Lambda 表达式也可以递归:

计算一个数的阶乘:`oper= lambda n: 1 if n==1 else n*oper(n-1)`

在 lambda 表达式中,表达式左边即为返回;

2. 命名函数与匿名函数的选择?

- 建议使用匿名函数的情况：函数逻辑简单，只在一个地方调用（常作为高阶函数的参数）
- 建议使用命名函数的情况：函数逻辑复杂，需要多步操作，需要多个地方重复使用或需要加文档说明的场景

2.2.3 变量类型注解

类型注解

- 类型注解是Python中的一种语法特性，用于明确标识变量、函数参数和返回值的数据类型，从而使代码更清晰、更安全、更易维护。

```
# 定义变量
a = 695
score = 98.5
hobby = "Python"
flag = True
pic = None

names = ["A", "C", "E"]
phones = {"13309091111", "15209109121"}
options = {"count": 0, "total": 0}
goods = ("手机", 5999, 1)
```

```
# 定义变量
a: int = 695
score: float = 98.5
hobby: str = "Python"
flag: bool = True
pic: None = None

names: list[str] = ["A", "C", "E"]
phones: set[str] = {"13309091111", "15209109121"}
options: dict[str, int] = {"count": 0, "total": 0}
goods: tuple[str, int, int] = ("手机", 5999, 1)
```

元组如果加注解,需要给每个项都加上注解;

Names: list[str | int]:一个元组可以用 | 来加入多个类型;

注意: 在对变量进行直接赋值,或者涉及到变量的运算、容器的推导等场景时,解释器会自动推导出变量的类型。

py 中有类型推断,故不必要时可以不加;

函数-类型注解

- 为函数添加类型注解,其实主要就是为函数的参数和返回值添加类型注解,具体语法如下:

```
def calc(scores: list[int]) -> float:
    return sum(scores) / len(scores)

def calc_data(scores: list[int]) -> tuple[int, int, float]:
    max_v = max(scores)
    min_v = min(scores)
    avg_v = sum(scores) / len(scores)
    return max_v, min_v, avg_v
```

返回值的类型

参数的类型

给*args 加注解时,由于 args 本身为 tuple,所以注解其项的类型即可;

3.0 模块

Python模块

- Python模块(module): 一个.py文件就是一个模块,模块是Python程序的基本组织单位。在模块中可以定义变量、函数、类,以及可执行的代码。



```
# 随机点名器
import random

# 人物列表
names = ["王林", "李慕婉",
         "许立国", "韩立",
         "涛哥", "莫厉海",
         "十三", "虎咆",
         "红蝶", "天运子"]
print(random.choice(names))
```

main.py

提高代码复用性 降低开发门槛 避免命名冲突

3.1 导入

模块导入方式

- 在使用模块中提供的功能之前，必须先导入，再使用。
- 导入模块的具体语法如下：

导入形式	代码样例	调用方式	调用方式
<code>import 模块名</code>	<code>import random, os</code>	模块名.功能名	<code>random.randint(10, 100)</code>
<code>import 模块名 as 别名</code>	<code>import random as rd</code>	别名.功能名	<code>rd.randint(10, 100)</code>
<code>from 模块名 import 功能名</code>	<code>from random import randint,choice</code>	功能名	<code>randint(10, 100)</code>
<code>from 模块名 import 功能名 as 别名</code>	<code>from random import randint as rint</code>	别名	<code>rint(10, 100)</code>
<code>from 模块名 import *</code>	<code>from random import *</code>	功能名	<code>randint(10, 100)</code>

功能1 功能2 功能3

功能4 功能5 功能6

导入模块 和 导入函数的区别:调用方式不同;

3.2 自定义模块

自定义模块

- 当开发一些复杂的项目，为了让项目结构更清晰，更便于项目的维护管理 及 代码的复用，可能会把一个项目拆分为若干个模块。

config.py

```
# 基础配置项
PAGE_TITLE = "AI聊天气", # 页面标题
PAGE_ICON = "🤖", # 页面icon
LAYOUT = "wide", # 页面布局, 可选值有: "centered" | "wide"
INITIAL_SIDEBAR_STATE = "expanded", # 侧边栏初始状态
HELP_URL = "https://www.extremelycoolapp.com/help"
SESSION_LOCATION = "sessions/" # 会话数据存储位置
```

session.py

```
# 加载会话列表
def load_session():
    sessions = []
    if os.path.exists("sessions"):
        for filename in os.listdir("sessions"):
            if filename.endswith(".json"):
                sessions.append(filename[:-5])
    return sorted(sessions, reverse=True)
.....
```

ai_partner.py

```
# 导入
from session import load_session, save_session

# 新建会话按钮
if st.button("新建会话", icon="🗨️", use_container_width=True):
    # 保存当前会话 (如果有)
    if st.session_state.current_session:
        save_session(st.session_state.current_session)
    # 创建新会话
    new_session_name = generate_new_session_name()
    st.session_state.current_session = new_session_name
    st.session_state.messages = []
    save_session(new_session_name)
    st.session_state.sessions = load_sessions()
    st.rerun()
```

注意：每一个python文件都可以作为一个模块，模块的名字就是文件的名字（建议使用python标识符定义，规范命名）。

py 中内置变量 `__name__`:当此文件运行时, `__name__` 值为 `__main__`,当导入时,此 py 的 `__name__` 值为文件名字,故可以作为主程序的区分;

将此 py 文件的运行代码放到 `if __name__=="__main__"` 块中,即可作为主程序;

__all__

__all__ 是一个模块级别的特殊变量，用于指定 `from 模块名 import *` 时会导入哪些功能（*通配了哪些功能）。

my.py

```
__all__ = ["log_separator1", "log_separator3", "PI"]

PI = 3.1415926
NAME = "黑马☆涛哥"

def log_separator1():
    print("-" * 30)

def log_separator2():
    print("+ " * 30)

def log_separator3():
    print("# " * 30)

def log_separator4():
    print("* " * 30)
```

test.py

```
from my import *

log_separator1()
log_separator3()
log_separator2()
log_separator4()

print(PI)
print(NAME)
```

注意： `__all__` 控制的是 `from ... import *` 时，要导入的功能，并不会影响直接导入具体的功能（如：`from ... import 功能`）。

用字符串列表进行通配;

3.3 包

包

- 包：本质就是一个文件夹，该文件夹中可以包含若干python模块（.py文件），文件夹下还包含了一个 `__init__.py`。
- 作用：模块文件较多时，用来管理多个模块。（包的本质也是一个模块）

与普通文件夹的区别:是否有 `__init__` 文件;

包的导入方式

导入形式	代码样例	调用方式	调用方式
<u>import 包名.模块名</u>	import utils.my_fun	包名.模块名.功能名	utils.my_fun.log_separator1()
from 包名 import 模块名	from utils import my_fun	模块名.功能名	my_fun.log_separator1()
from 包名 import *	from utils import *	模块名.功能名	my_fun.log_separator1()
from 包名.模块名 import 功能名	from utils.my_fun import log_separator1	功能名	log_separator1()
from 包名.模块名 import *	from utils.my_fun import *	功能名	log_separator1()

注意： 在通过 `'from 包名 import *'` 导入全部模块的时候，需要在 `__init__.py` 文件中添加 `'__all__=[]'`，控制允许导入的模块列表。

```
# 描述包信息
__version__ = "1.0.0"
__author__ = "黑马-涛哥"
```

__init__ 中写入描述包的信息;__all__描述*通配符所匹配模块

__all__ 以模块字符串名称的列表来写;

导入时找包会从文件当前所在目录同级目录来找;

若包和当前所在目录不在同一目录下,则要用绝对路径(即项目目录.所在目录.包.模块);

4.0 py 的面向对象

面向过程编程

- 核心思想: 把一个需求分解成一系列要执行的步骤, 然后按照步骤依次执行这些任务(关注的是流程、步骤)。
- 适用场景: 面向过程编程非常直接, 适合简单、线性的任务。



面向对象编程

- 对象可以理解为现实中具体的人/物在程序中的数字化身(万物皆对象)。
- 它把一个人/物的特征和功能打包到一起, 是面向对象编程的基本单元。(关注的是谁来帮我做这件事儿)



类:描述的是一组具有相同属性(特征)和方法(功能/行为)的模板。

对象:对象是类的实例, 是基于类创建出来的(实例对象)。

对象是由类创建出来的, 创建对象的过程, 也称为对象的实例化。一个类可以创建无数个对象。

4.1 类和对象

类的定义

- 定义类的语法如下：

```
# 定义类
class 类名:
    pass

# 创建对象
对象名 = 类名()
对象名.属性名1 = 属性值1
对象名.属性名2 = 属性值2
```

```
# 定义类
class Car:
    pass

# 创建对象
c1 = Car()
c1.brand = "BMW"
c1.name = "X5"
c1.price = 500000
print(c1.__dict__)
```

说明：类名的命名规范，遵循大驼峰命名法，每个单词的首字母都是大写，单词之间没有分隔符，比如：UserInfo，UserAccount。

说明：__dict__是Python中用户自定义类实例的一个特殊属性，用于以字典形式存储对象的属性。

py 中的属性可以动态的添加到类中(不够严谨);

类的定义

- 定义类的语法如下：

```
# 定义类
class 类名:
    def __init__(self, 参数列表):
        self.属性名 = 参数值
        self.属性名 = 参数值

# 创建对象
对象名 = 类名(参数列表)
```

```
# 定义类
class Car:
    def __init__(self, c_brand, c_name, c_price):
        self.brand = c_brand
        self.name = c_name
        self.price = c_price

# 创建对象
c1 = Car("BMW", "X5", 500000)
print(c1.__dict__)
```

self: 方法的第一个参数，表示当前创建的实例对象 **this指针**

__init__: 初始化方法，对象创建后自动调用，主要用于设置对象的初始状态（设置对象属性）

Py 中也可以不写构造函数(自动继承父类的构造函数)

4.2 类的方法

实例方法

- 在类中定义实例方法时，定义语法与之前学习的函数定义的方式是一致的。

```
# 定义类
class 类名:
    def __init__(self, 形参列表):
        self.属性名 = 参数值
        self.属性名 = 参数值

    def 方法名(self, 形参列表):
        ...

    def 方法名(self, 形参列表):
        ...

# 创建对象
对象名 = 类名(参数列表)
对象名.方法名(实参)
```

```
class Car:
    def __init__(self, brand, name, price):
        self.brand = brand
        self.name = name
        self.price = price

    def running(self):
        print(f"{self.brand} {self.name} 正在高速行驶...")

    def total_cost(self, discount, rate):
        return self.price * discount + self.price * rate

c1 = Car("BMW", "X5", 500000)
total_cost = c1.total_cost(0.9, 0.1)
print(f"提车总价为: {total_price:.0f}")

c1.running()
```

self表示当前实例对象，方法调用时无需传递

与函数定义区别:第一个参数必须为 `self`(调用时不用传递)
其他定义如(不定长参数,默认参数,类型注解等)与普通函数相同;

4.3 魔法方法

魔法方法

- 魔法方法是指Python中提供的以双下划线开头和结尾的特殊方法，用于定义类的特殊行为，比如：`__init__`。
- 魔法方法是不需要我们手动调用的，Python会在合适的时机自动调用。

魔法方法	描述
<code>__init__</code>	初始化方法
<code>__str__</code>	字符串表示的方法
<code>__eq__</code>	比较两个对象是否相等 (equal)
<code>__lt__</code> , <code>__le__</code> , <code>__gt__</code> , <code>__ge__</code>	支持比较两个对象的大小(小于(<code>less than</code>), 小于等于(<code>less than or equal</code>), 大于(<code>greater than</code>), 大于等于(<code>greater than or equal</code>))

`c1==c2`:由于类名实际为地址,所以直接比较的是地址值,必然为 `false`;

```
class Car:
    def __init__(self, brand, name, price):
        self.brand = brand
        self.name = name
        self.price = price
    def running(self):
        print(f"{self.brand} {self.name} 正在高速行驶...")
    def __str__(self):
        return f"{self.brand} {self.name} {self.price}"
    def __eq__(self, other):
        return self.price == other.price and self.brand == other.brand and self.name == other.name
    def __lt__(self, other):
        return self.price < other.price

c1 = Car("BMW", "X5", 500000)
print(c1)      BMW X5 500000
c2 = Car("BMW", "X5", 500000)
print(c2)      BMW X5 500000

print(c1 == c2)  True
print(c1 < c2)   False
```

定义了魔法方法后,可进行比较;

4.4 类的属性

属性

- 属性分为：
 - 实例属性：实例属性属于每个具体对象的属性，每个对象都是独立的。（各个对象特有的数据）
 - 类属性：类属性是属于类本身的属性，所有实例共享的。（所有对象共享的数据或配置）

```
class Car:
    wheel = 4 # 轮胎数量
    tax_rate = 0.1 # 购置税
    def __init__(self, c_brand, c_name, c_price):
        self.brand = c_brand
        self.name = c_name
        self.price = c_price

    def running(self):
        print(f'{self.brand} {self.name} 正在高速行驶...')

c1 = Car("BYD", "汉", 180000)
c2 = Car("Tesla", "Model Y", 260000)
```

说明：通过实例查找属性时，会先查找实例属性，实例属性不存在时，再查找类属性。

实例对象.属性也可访问到 类属性；
当实例属性和类属性同名时,用实例对象访问优先访问实例属性；

5.0 异常

```
try:
    print("=====")
    # print(my_name)
    # print(1 / 0)
    # print("ABC"[10])
    print("ABC".hello)
    print("=====")
except NameError as e: # 捕获的是 NameError 类型的异常
    print("名字不存在，请检查变量或函数名字，异常信息：", e)
except ZeroDivisionError as e:
    print("0不能做被除数，异常信息：", e)
except IndexError as e:
    print("索引错误，异常信息：", e)
except Exception as e: # 捕获所有的异常
    print("程序运行出错了，请联系管理员，错误信息：", e)
finally: # 无论程序是否正常运行，finally代码块中的代码都会运行
    print("资源释放 ~")
```

用 try...except 来为程序错误做预案；
Exception 可以捕获所有类型的异常；
Finally 是无论程序是否错误,都会执行的程序,一般用于释放资源(如关闭文件);

异常的传递

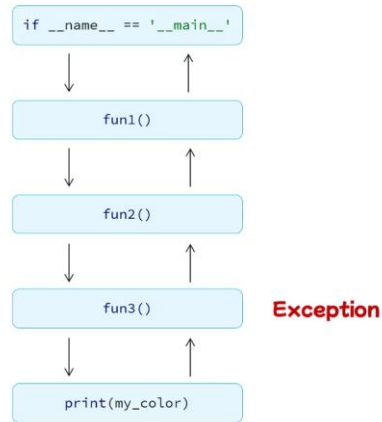
- 异常传递就是异常在函数调用中层层上报的过程，直到有人处理它，或者程序崩溃。

```
def fun1():
    print("fun1 ... running ...")
    fun2()

def fun2():
    print("fun2 ... running ...")
    fun3()

def fun3():
    print("fun3 ... running ...")
    print(my_color)

if __name__ == '__main__':
    fun1()
```



指的是 多层函数调用,若上层函数异常,则异常会层层回传,直到最底层;
故在最底层(即主函数)中直接处理异常较好(因为返回后,下层无法处理上层的错误),在最底层可以一次处理完所有的错误;
捕获完异常后,会继续向下执行代码,故捕获异常的位置很重要,涉及到程序如何继续执行;

6.0 py 中的文件操作

文件操作入门

- 日常我们操作文件时，基本分为三步操作：打开、读/写、关闭。

```
# 1. 打开文件
f = open("resources/望庐山瀑布.txt", "r", encoding="utf-8")

# 2. 读取文件
content = f.read()
print(content)

# 3. 关闭文件
f.close()
```

```
# 1. 打开文件
f = open("resources/静夜思.txt", "w", encoding="utf-8")

# 2. 写入文件
f.write("窗前明月光，\n")
f.write("疑是地上霜，\n")
f.write("举头望明月，\n")
f.write("低头思故乡，\n")

# 3. 关闭文件
f.close()
```

编码：是将字符(文字、数字、符号)转换为计算机能够存储和处理的数字代码的规则系统，如：ASCII、GBK、UTF-8。

注意：如果操作完文件，并未调用close方法关闭文件，同时程序没有停止运行，那么这个文件将一直被Python程序占用，无法操作。

文件操作

```

# 读文件
with open("resources/望庐山瀑布.txt", "r", encoding="utf-8") as f:
    content = f.read()
    print(content)

# 写文件
with open("resources/静夜思.txt", "w", encoding="utf-8") as f:
    f.write("静夜思(李白)\n\n")
    f.write("窗前明月光, \n")
    f.write("疑是地上霜, \n")
    f.write("举头望明月, \n")
    f.write("低头思故乡, \n")

```

路径写法:

- 相对路径: 相对于当前工作目录的路径;
- 绝对路径: 是从文件系统的目录开始, 完整的描述文件位置的路径;

操作模式:

- r: 只读的方式打开文件, 指针放在文件的开头。(默认模式)
- w: 只写入模式, 从头编辑, 原有内容会被删除; 文件不存在则创建新文件
- a: 追加模式, 新内容会被追加在原有内容之后; 文件不存在则创建新文件

```

f = open("resources/望庐山瀑布.txt", "r", encoding="utf-8")

# 2. 读取文件内容
# content = f.read() # 读取所有内容
# print(content)

content_list = f.readlines()
for line in content_list:
    print(line.strip())

# 3. 关闭文件
f.close()

```

.read()一次读取所有;.readline()读取一行;.readlines()返回每行组成的列表;
写的时候,若文件不存在会直接创建文件;w 模式会进行覆盖;

文件操作-资源释放

● 资源释放方案:

```

# 1. 打开文件
f = open("resources/静夜思.txt", "w", encoding="utf-8")

try:
    # 2. 写入文件
    f.write("静夜思(李白)\n\n")
    f.write("窗前明月光, \n")
    f.write("疑是地上霜, \n")
    f.write("举头望明月, \n")
    f.write("低头思故乡, \n")
finally:
    # 3. 关闭文件
    f.close()

```

繁琐

```

with open("resources/静夜思.txt", "w", encoding="utf-8") as f:
    f.write("静夜思(李白)\n\n")
    f.write("窗前明月光, \n")
    f.write("疑是地上霜, \n")
    f.write("举头望明月, \n")
    f.write("低头思故乡, \n")

```

推荐

提示: with语句(上下文管理器)的核心作用就是确保资源的总是被正确获取和释放(即使发生异常, 也会被正确释放), 也是项目开发中的推荐方式。

避免文件无法关闭的两种方式(推荐第二种,不用写 close);

读写json格式文件

- JSON是软件开发中最常用的数据交换格式，而为了简化JSON数据的处理，在Python标准库中就提供了处理JSON数据的核心模块 json。



The image shows two screenshots of a Python IDE. The top screenshot, titled '写入json数据' (Write JSON data), shows a Python script that creates a dictionary object and writes it to a file named 'session.json' using the json.dump() function. The bottom screenshot, titled '读取json数据' (Read JSON data), shows a Python script that reads the data from 'session.json' using the json.load() function and prints it.

```
import json

obj = {
    "name": "张三",
    "age": 18,
    "gender": "男",
    "hobbies": ["reading", "swimming"]
}

with open("resources/session.json", "w", encoding="utf-8") as f:
    json.dump(obj, f, ensure_ascii=False, indent=2)
```

序列化

```
import json

with open("resources/session.json", "r", encoding="utf-8") as f:
    obj = json.load(f)
    print(obj)
```

反序列化

Dump 函数参数:ensure_ascii:是否使用 ascii 码(由于 ascii 不能编码中文,设为 false 即可)
Indent:每行数据前的缩进个数;设置缩进使得格式更好看;